

Fragment Parsing

March 2026

1 Background

1.1 Context-Free Grammars (copied from satta1994)

A *context-free grammar* (CFG) is a four-tuple $G = (N, \Sigma, P, S)$, where N is a finite set of nonterminal symbols, Σ is a finite set of terminal symbols with $N \cap \Sigma = \emptyset$, P is a finite set of productions, and $S \in N$ is the start symbol.

Productions in P have the form $D_r \rightarrow Z_{r,1}Z_{r,2}\dots Z_{r,\pi_r}$, where $D_r \in N$ and $Z_{r,j} \in N \cup \Sigma$, $\pi_r \geq 0$. A production is *null* if $\pi_r = 0$ and a *chain* production if $\pi_r = 1$ and $Z_{r,1} \in N$.

The language derived by G is $L(G) = \{w \mid S \xrightarrow{*} w, w \in \Sigma^*\}$.

1.2 Decomposition

Let ω be an input string that can be decomposed as

$$\omega = \alpha \cdot \beta \cdot \gamma$$

where $\alpha, \gamma \in \Sigma^*$ and $\beta \in \Sigma \cup N$.

There exists a derivation

$$\alpha\beta\gamma \Rightarrow \alpha A \gamma$$

or generally,

$$\alpha\beta\gamma \Rightarrow [\alpha/x]A[\gamma/y]$$

where $x, y \in \Sigma^* \cup N^* \cup \{\varepsilon\}$. The notation $[\alpha/x]$ denotes a correspondence between the input flank α and the production symbol x — specifically, $x \xrightarrow{*} \alpha$ (x derives α). The precise treatment of the recursive case is given in Section 3.

The possible values for the input flanks (α, γ) and production flanks (x, y) are:

I — input flanks (α, γ)	II — production flanks (x, y)
(1) $\alpha = \varepsilon, \gamma = \varepsilon$	(1) $x = \varepsilon, y = \varepsilon$
(2) $\alpha = \varepsilon, \gamma \in \Sigma^+$	(2) $x = \varepsilon$
(3) $\alpha \in \Sigma^+, \gamma = \varepsilon$	(3) $y = \varepsilon$
(4) $\alpha, \gamma \in \Sigma^+$	(4) $x, y \neq \varepsilon$

1.3 Context Reduction Matrix

Mixing and matching cases from I and II gives all possible reduction scenarios for β . Given a production $A \rightarrow x\beta y$ and input fragment $\alpha\beta\gamma$, the grammar defines what flanks are expected. A flank is a *gap* if the production requires it but the input does not supply it.

Two notations are used in the derivations below:

- (x) — x is a production symbol required by the grammar but absent from the input (a gap)
- $[\alpha/x]$ — α is present in the input and corresponds to production symbol x

Cases 1.x — $\alpha = \varepsilon, \gamma = \varepsilon$:

(1.1)	$\beta \rightarrow A$
(1.2)	$\beta \rightarrow \beta(y) \rightarrow A$
(1.3)	$\beta \rightarrow (x)\beta \rightarrow A$
(1.4)	$\beta \rightarrow (x)\beta(y) \rightarrow A$

Cases 2.x — $\alpha = \varepsilon, \gamma$ present:

(2.1)	$\beta\gamma \rightarrow A\gamma$
(2.2)	$\beta\gamma \rightarrow \beta[\gamma/y] \rightarrow A\gamma'$
(2.3)	$\beta\gamma \rightarrow (x)\beta\gamma \rightarrow A\gamma$
(2.4)	$\beta\gamma \rightarrow (x)\beta[\gamma/y] \rightarrow A\gamma'$

Cases 3.x — α present, $\gamma = \varepsilon$:

(3.1)	$\alpha\beta \rightarrow \alpha A$
(3.2)	$\alpha\beta \rightarrow \alpha\beta(y) \rightarrow \alpha A$
(3.3)	$\alpha\beta \rightarrow [\alpha/x]\beta \rightarrow \alpha' A$
(3.4)	$\alpha\beta \rightarrow [\alpha/x]\beta(y) \rightarrow \alpha' A$

Cases 4.x — α present, γ present:

(4.1)	$\alpha\beta\gamma \rightarrow \alpha A\gamma$
(4.2)	$\alpha\beta\gamma \rightarrow \alpha\beta[\gamma/y] \rightarrow \alpha A\gamma'$
(4.3)	$\alpha\beta\gamma \rightarrow [\alpha/x]\beta\gamma \rightarrow \alpha' A\gamma$
(4.4)	$\alpha\beta\gamma \rightarrow [\alpha/x]\beta[\gamma/y] \rightarrow \alpha' A\gamma'$

1.3.1 Note on Null Productions

We assume β is never nullable. For nullable flanks (x or y), an implementation may precompute the nullable set $\mathcal{N}_G = \{A \in N \mid A \xrightarrow{*} \varepsilon\}$ and add skip-rules: for any case where a flank symbol is in $\mathcal{N}_G$, add the corresponding case with that flank dropped. For example, if $x \in \mathcal{N}_G$, case 4.3 ($[\alpha/x]\beta\gamma \rightarrow \alpha' A\gamma$) admits a skip to case 2.1 ($\beta\gamma \rightarrow A\gamma$) directly. Epsilon productions need not fire at runtime.

1.4 Recurrence

Let $R(\omega) \subseteq N$ denote the set of nonterminals that span ω . The recurrence is defined as follows:

Base case:

$$R(\beta) = \{A \in N \mid A \rightarrow x\beta y \in P, \\ x, y \in N \cup \Sigma \cup \{\varepsilon\}\}$$

where x and y are either matched against input flanks or treated as gaps per the CR matrix. This is a fixed-point computation over a finite set — cycles in the grammar do not cause non-termination.

Inductive step: For $\omega = \alpha\beta\gamma$, apply the CR matrix to obtain $\alpha' A\gamma'$ for each $A \in R(\beta)$:

$$R(\omega) = \bigcup_{\{A \in R(\beta)\}} R(\alpha' A\gamma')$$

where $|\alpha' A\gamma'| < |\omega|$, guaranteeing termination.

A *final* nonterminal F is any $A \in R(\omega)$ such that A spans the entire input — i.e. no unreduced symbols remain. F need not be the start symbol S ; any nonterminal covering ω constitutes a valid parse result.

2 Algorithm

2.1 Recognition Table

Let $\omega = w_1 w_2 \dots w_n$ be the input string. The *recognition table* T is an $(n+1) \times (n+1)$ array of sets, where $T[i, j]$ contains *items* representing partial or complete derivations over the span $[i, j]$ (i.e. over $w_{i+1} \dots w_j$).

There are two kinds of items:

- A *complete item* I_A asserts that nonterminal A derives $w_{i+1} \dots w_j$.
- A *partial item* $I_r^{s,t}$ asserts that production r has been partially assembled: symbols $Z_{\{r,s+1\}} \dots Z_{\{r,t\}}$ (the head-adjacent slice) derive $w_{i+1} \dots w_j$, and the remaining symbols $Z_{\{r,1\}} \dots Z_{\{r,s\}}$ and $Z_{\{r,t+1\}} \dots Z_{\{r,\pi_r\}}$ are yet to be combined.

2.2 H-Cover

The *H-cover* $\mathcal{H}(G)$ pre-computes all valid item-to-item relationships from the grammar. It consists of:

- **Projections** (I_A, ξ) : item I_A can be projected from child ξ , where ξ is either a terminal or another item.
- **Left expansions** (I, ξ, I') : I' is the trigger — the known right child already in the table. ξ is the left child to be found elsewhere in the table. When both are present, the result I spans further left than I' .
- **Right expansions** (I, I', ξ) : I' is the trigger — the known left child already in the table. ξ is the right child to be found elsewhere in the table. When both are present, the result I spans further right than I' .

The cover is computed once per grammar and reused across all inputs.

2.3 Agenda Algorithm

The algorithm maintains a queue (agenda) of items to process. When item I' is added to $T[i, j]$ for the first time, it is pushed onto the agenda.

Processing item I' at $T[i, j]$ applies four rules:

1. **Project**: for each $(I, I') \in \text{projections}$, add I to $T[i, j]$.
2. **Left-expand**: for each $(I, \xi, I') \in \text{left-expansions}$, find all $i' \leq i$ such that $\xi \in T[i', i]$, and add I to $T[i', j]$.
3. **Right-expand**: for each $(I, I', \xi) \in \text{right-expansions}$, find all $j' \geq j$ such that $\xi \in T[j, j']$, and add I to $T[i, j']$.
4. **Reverse lookups**: In rules 2 and 3, I' is the trigger and ξ is what we look for. Rule 4 handles the symmetric case: I' is the child that was being looked for. Scan the table for items that could serve as the trigger, and for each match add the result I .

Example. Take $A \rightarrow BC$ with head C . The cover gives left-expansion $I_A \rightarrow I_B I_r^{(1,2)}$, where $I_r^{(1,2)}$ is the known right child and I_B is ξ .

- **Rule 2 (left-expand)**: $I_r^{(1,2)}$ arrives at $T[1, 2]$. Scan left for I_B at $T[0, 1]$. If found, add I_A to $T[0, 2]$.

- **Rule 4 (reverse):** Suppose I_B arrives at $T[0, 1]$ after $I_r^{(1,2)}$ was already processed — rule 2 already fired and found nothing. Now I_B is the trigger. It is the ξ for the left-expansion above, so scan *right* for $I_r^{(1,2)}$ at $T[1, j]$. Found at $T[1, 2]$. Add I_A to $T[0, 2]$.

Rules 2 and 4 together ensure I_A is found regardless of which child arrives first. The same pairing holds for right-expand (rule 3) and its reverse.

The algorithm terminates because T is finite and each item is enqueued at most once.

2.4 Correctness

Theorem: If there exists a nonterminal $A \in N$ and a derivation tree rooted at A whose yield (with zero or more leaves replaced by virtual gap symbols) equals ω , then a valid item appears in $T[0, n]$ after the algorithm completes.

Fundamental assumption: fix a derivation tree $\mathcal{T}$ witnessing the theorem — i.e. $A \xRightarrow{*} \omega'$ where ω' equals ω with zero or more positions replaced by gaps. The proof shows that at each step the algorithm tracks $\mathcal{T}$: every prefix (or suffix) subtree of $\mathcal{T}$ that spans the processed portion of the input is represented by an item in the table.

The proof proceeds in two parts, one for each fill direction.

2.4.1 Part 1 — Forward (prefix) induction

Claim: For every k , $1 \leq k \leq n$, $T[0, k]$ contains an item corresponding to the prefix subtree of $\mathcal{T}$ that spans $w_1 \dots w_k$.

Base ($k = 1$): Terminal w_1 seeds its projection items via the cover. If the prefix subtree of $\mathcal{T}$ at w_1 requires constituents to the left that are absent from the input, boundary stuffing injects the corresponding item into $T[0, 1]$ with those constituents marked virtual. Either way, the item for the prefix subtree exists in $T[0, 1]$.

Step: Assume $T[0, k]$ contains an item b for the prefix subtree spanning $w_1 \dots w_k$. Terminal $w_{\{k+1\}}$ seeds $T[k, k+1]$. Two cases:

1. b directly combines with $w_{\{k+1\}}$: right-expand fires and adds the combined item to $T[0, k+1]$.
2. b cannot combine directly: the derivation tree $\mathcal{T}$ requires more structure to the left of b before the combination is possible — i.e. b is the right child of some cover rule whose left child is absent from the input. The inductive fill reduces b leftward by injecting that left child as a virtual gap, producing a new item b' in $T[0, k]$. This repeats until an item emerges that can combine with $w_{\{k+1\}}$ via right-expand. Since $\mathcal{T}$ guarantees such a combination exists, termination is assured. $\square$

2.4.2 Part 2 — Backward (suffix) induction

Triggered only if $T[0, n]$ is empty after Part 1 — i.e. no full-span item was reached from the left alone.

Claim: For every k , $0 \leq k \leq n-1$, $T[k, n]$ contains an item corresponding to the suffix subtree of $\mathcal{T}$ that spans $w_k \dots w_n$.

Base ($k = n - 1$): Terminal w_n seeds its projection items. If the suffix subtree requires constituents to the right that are absent, boundary stuffing injects the item with those constituents virtual.

Step: Assume $T[k, n]$ contains an item b for the suffix subtree spanning $w_k \dots w_n$. Terminal w_k seeds $T[k - 1, k]$. Two cases:

1. b directly combines with w_k : left-expand fires and adds the combined item to $T[k - 1, n]$.
2. b cannot combine directly: $\mathcal{T}$ requires more structure to the right of b before the combination is possible. The inductive fill reduces b rightward by injecting the missing right child as a virtual gap, producing b' in $T[k, n]$, until an item emerges that combines with w_k via left-expand. $\mathcal{T}$ guarantees such a combination exists. $\square$

2.4.3 Convergence

After both passes, a final closure is run on $T[0, n]$ — applying the same fill logic to whatever the two inductions deposited there. Any item reachable by combining prefix and suffix derivations is found at this step.

Together, Parts 1 and 2 guarantee that $T[0, n]$ contains a valid item whenever there exists $A \in N$ and a derivation $A \xRightarrow{*} \omega'$ where ω' equals ω with zero or more symbols replaced by gaps — regardless of which direction the fragment is approached from.

Remark (sequential vs. simultaneous passes). The backward pass is an implementation choice — it fires only if $T[0, n]$ is empty after the forward pass. A simultaneous version (running both directions together, with items from each pass feeding the other) is possible and may find derivations that neither pass finds in isolation. The interaction between the two directions is currently unanalysed.

2.5 Tree Pruning

Recognition populates $T[0, n]$ with items and their derivations. Tree reconstruction follows derivation pointers back through the table, producing a parse forest. For ambiguous grammars or fragment inputs this forest can be exponentially large.

Open problem: the forest must be pruned to be useful. Two criteria are relevant:

1. **Gap minimisation:** trees with fewer virtual (dropped) constituents are better parses of the fragment — a tree with no gaps is a complete parse. Trees should be ranked or filtered by gap count.
2. **Structural plausibility:** among trees with equal gap count, some are more plausible parses than others depending on where the gaps fall (e.g. a missing right argument is more natural than a missing head). A scoring scheme over gap positions is needed.

Neither criterion is currently enforced — the implementation returns all trees up to a fixed cap. A principled pruning strategy is future work.